

DOCKER

Installation, container management

Virtualizzazione e integrazione di sistemi

Vittorio Ghini

a.a. 2024-25

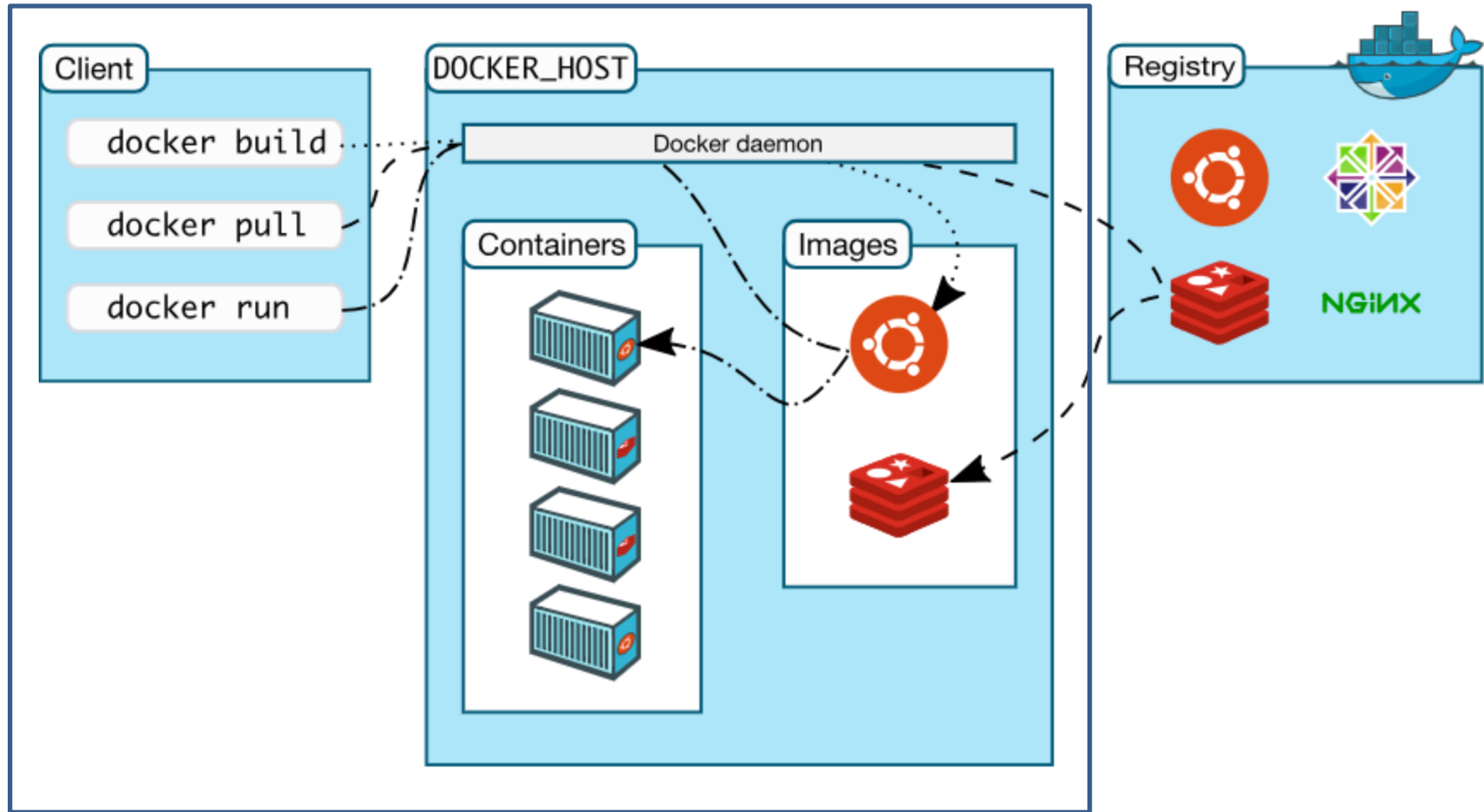
Ingegneria e scienze informatiche

Università di Bologna

Contents

- 1. Installing Docker
- 2. Executing Docker without sudo
- 3. Using the Docker command
- 4. Working with Docker images
 - Search, Load, Remove Docker images
- 5. Running a Docker container
 - Exiting Docker container does not remove container
 - Running another container
 - Running bash command in a container
- 6. Managing Docker containers
 - Run, Stop, Start, Remove container
- 7. Commit changes in a container to a docker image
- 8. Pushing Docker container to a Docker repository

Docker Architecture



1. Installing Docker (0/0)

- Qui una spiegazione dei tre metodi possibili (2023)
- <https://kinsta.com/it/blog/installare-docker-ubuntu/>
- **Installare Docker Con il Repository apt**
- `sudo apt update`
- `sudo apt install ca-certificates curl gnupg lsb-release "apt-transport-https" "software-properties-common"`
- registrate il portachiavi GPG di Docker con apt. Questo permetterà a apt di convalidare i pacchetti Docker installati.
- `sudo mkdir -p /etc/apt/keyrings`
- Scarica la chiave GPG di Docker per Ubuntu, la converte in una codifica OpenPGP standard e la salva nella directory keyring di apt
- `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`
- `sudo chmod a+r /etc/apt/keyrings/docker.gpg`
- Rileva architettura di sistema, identifica repository appropriato, il repository viene aggiunto come un nuovo elenco di pacchetti all'interno del file di apt `/etc/apt/sources.list.d/docker.list`
- `echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null`
- `sudo apt update`
- `sudo apt install docker-ce docker-ce-cli containerd.io`
- `sudo systemctl status docker`

1. Installing Docker on debian (1/1)

- Analogously, on debian os:
 - ✓ `sudo apt-get update`
 - ✓ `sudo apt install apt-transport-https ca-certificates curl`
 - ✓ `software-properties-common`
 - ✓ `curl -fsSL https://download.docker.com/linux/debian/gpg | sudo apt-key add -`
 - ✓ `sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/debian $(lsb_release -cs) stable"`
 - ✓ `sudo apt-get update`
 - ✓ `sudo apt-get install docker-ce docker-ce-cli containerd.io`
 - ✓ `sudo systemctl status docker`

2. Executing Docker without sudo

- By default, the docker command can only be run the root user or by a user in the **docker** group, which is automatically created during Docker's installation process. If you attempt to run the docker command without prefixing it with sudo or without being in the docker group, you'll get an output like this:
- If you want to avoid typing sudo whenever you run the docker command, add your username to the docker group:

```
sudo usermod -aG docker ${USER}
```

- for example: `sudo usermod -aG docker vic`

- Confirm that your user is now added to the docker group by typing:

```
id -nG
```

OUTPUT: vic adm cdrom sudo dip plugdev lpadmin sambashare

- reload user

```
su - ${USER}
```

for example: `su - vic`

3. Using the Docker command

- general syntax: `docker [option] [command] [arguments]`
- To view all available subcommands, type: `docker`
- Options:
 - `image` Manage images
- Commands:
 - `attach` Attach local standard input, output, and error streams to a running container
 - `commit` Create a new image from a container's changes
 - `cp` Copy files/folders between a container and the local filesystem
 - `exec` Run a command in a running container
 - `images` List images
 - `login` Log in to a Docker registry
 - `logout` Log out from a Docker registry
 - `ps` List containers
 - `pull` Pull an image or a repository from a registry
 - `push` Push an image or a repository to a registry
 - `restart` Restart one or more containers
 - `rm` Remove one or more containers
 - `rmi` Remove one or more images
 - `run` Run a command in a new container
 - `search` Search the Docker Hub for images
 - `start` Start one or more stopped containers
 - `stop` Stop one or more running containers
 - `tag` Create a tag `TARGET_IMAGE` that refers to `SOURCE_IMAGE`
 - `update` Update configuration of one or more containers
 - `version` Show the Docker version information

4. Working with Docker images (1/2)

- Docker containers are built from Docker images. By default, Docker pulls these images from Docker Hub, a Docker registry managed by Docker, the company behind the Docker project. Anyone can host their Docker images on Docker Hub, so most applications and Linux distributions you'll need will have images hosted there.
- You can download a container image and run it, by using the run command.
- To check whether you can access and download images from Docker Hub, type:

```
docker run hello-world
```

- The output will indicate that Docker is working correctly:

```
OutputUnable to find image 'hello-world:latest' locally
```

```
latest: Pulling from library/hello-world
```

```
9bb5a5d4561a: Pull complete
```

```
Digest:
```

```
sha256:3e1764d0f546ceac4565547df2ac4907fe46f007ea229fd7ef2718514bcec35d
```

```
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

4. Working with Docker images (2/2)

- Docker was initially unable to find the hello-world image locally, so it downloaded the image from Docker Hub, which is the default repository.
- Once the image downloaded, Docker created a container from the image and the application within the container executed, displaying the message.
- The downloaded images is copied into a local storage.

4.1. Searching&Loading Docker images

- You can search for images available on Docker Hub by using the docker command with the search subcommand. To search for the Ubuntu image, type:

```
docker search ubuntu
```

- The script will crawl Docker Hub and return a listing of all images whose name match the search string. In this case, the output will be similar to this:

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
ubuntu	Ubuntu is a Debian-based Linux operating sys...	9336		[OK]
dorowu/ubuntu-desktop-lxde-vnc	Docker image to provide HTML5 VNC interface ...	283		[OK]
rastasheep/ubuntu-sshd	Dockerized SSH service, built on top of offi...	209		[OK]
consol/ubuntu-xfce-vnc	Ubuntu container with "headless" VNC session...	165		[OK]
ansible/ubuntu14.04-ansible	Ubuntu 14.04 LTS with ansible	96		[OK]

- In the OFFICIAL column, OK indicates an image built and supported by the company behind the project. Once you've identified the image that you would like to use, you can download it to your computer using the pull subcommand.
- Execute the following command to download the official ubuntu image to your computer:

```
docker pull ubuntu
```

- After an image has been downloaded, you can then run a container using the downloaded image with the run subcommand.
- To see the images that have been downloaded to your computer, type:

- ```
docker images
```

## 4.2. Remove local Docker images

- To see the images saved in your computer, type:
- `docker images`
- You can remove the images in your computer by using the docker command `remove image rmi`. The `rmi` docker command requires a list of images to be removed.
- The generic syntax is:

`docker rmi imagename1 imagename2 .... imagenameN`

- For example, to remove the `ubuntu` and `httpd` docker images, type:  
`docker rmi ubuntu httpd`

# 5. Running a Docker container

- Containers can be interactive. As an example, let's run a container using the latest image of Ubuntu. The combination of the `-i` and `-t` switches gives you interactive shell access (using `stdin/sdtou/stderr`) into the container:
  - `docker run -it ubuntu`
  - or
  - `docker run -it --name myubuntu ubuntu`
- where the option `--name myubuntu` assigns the name `myubuntu` to the container. If no option `--name` is provided, docker creates and assign a random name to the container.
- Your command prompt should change to reflect the fact that you're now working inside the container and should take this form:

```
root@d9b100f2f636:/#
```
- Note the container id in the command prompt. In this example, it is `d9b100f2f636`. You'll need that container ID later to identify the container when you want to remove it.

# 5.1 Running commands in a container

- Now you can run any command inside the myubuntu container. For example, let's update the package database inside the container. You don't need to prefix any command with `sudo`, because you're operating inside the container as the root user:
- Check if the package `node.js` already exist in your system, but it does not exist

```
node -v
```

```
bash: node: command not found
```

- Install the `node.js` package.

```
apt update
```

- Then install any application in it. Let's install `Node.js`:

```
apt install nodejs
```

- This installs `Node.js` in the container from the official Ubuntu repository. When the installation finishes, verify that `Node.js` is installed:

```
node -v
```

- You'll see the version number displayed in your terminal:      `Outputv8.10.0`
- Any changes you make inside the container only apply to that container.
- To exit the container, type `exit` at the prompt.
- If you restart the container, the previous `node.js` installation has been lost.
- Try again "`docker run -it ubuntu`" and then run `node -v`

# 6. Managing Docker containers

- After using Docker for a while, you'll have many active (running) and inactive containers on your computer. To view the active ones, use:

- `docker ps`

- You will see output similar to the following:

| CONTAINER ID | IMAGE | COMMAND | CREATED |
|--------------|-------|---------|---------|
| ....         | ....  | ....    | ....    |

- If you started two containers; one from the hello-world image and another from the ubuntu image. Both containers are no longer running, but they still exist on your system.
- To view all containers — active and inactive, run `docker ps` with the `-a` switch:

- `docker ps -a`

- You'll see output similar to this:

| CONTAINER ID | IMAGE       | COMMAND     | CREATED                                          | NAME                    |
|--------------|-------------|-------------|--------------------------------------------------|-------------------------|
| d9b100f2f636 | ubuntu      | "/bin/bash" | About an hour ago Exited (0) 8 minutes ago       | <b>myubuntu</b>         |
| 01c950718166 | hello-world | "/hello"    | About two hours ago Exited (0) About an hour ago | <b>festive_williams</b> |

- To view the latest container you created, pass it the `-l` switch:

`docker ps -l`

| CONTAINER ID | IMAGE  | COMMAND     | CREATED                                    |
|--------------|--------|-------------|--------------------------------------------|
| d9b100f2f636 | ubuntu | "/bin/bash" | About an hour ago Exited (0) 8 minutes ago |

# 6.1. Stop & Start Docker containers

- To stop a running container, use `docker stop`, followed by the **container ID** or **container name**, in a different shell of the hosting host.
  - `docker stop myubuntu`
  - `docker stop d9b100f2f636`
- To start a stopped container, use `docker start`, followed by the container ID or the container's name. Let's start the Ubuntu-based container with the ID of `d9b100f2f636`:
  - `docker start d9b100f2f636`
- or
  - `docker start -ia d9b100f2f636`
- where the flag `-i` attach container's STDIN
- and the flag `-a` attach STDOUT/STDERR and forward signals
- The container will start, and you can use `docker ps` to see its status:

## 6.2 Remove Docker containers

- Once you've decided you no longer need a container anymore, remove it with the docker **rm** command, again using either the **container ID** or the **name**. Use the docker ps -a command to find the container ID or name for the container associated with the hello-world image and remove it.

```
docker rm festive_williams
```

```
docker rm ec68819568a3
```

# 7. Committing Changes in a Container

## creating a new Docker Image (1/2)

- When you start up a Docker image, you can create, modify, and delete files just like you can with a virtual machine. The changes that you make will only apply to that container. You can start and stop it, but once you destroy it with the `docker rm` command, the changes will be lost for good.
- Fortunately, you can save the state of a container as a new Docker image.
- As an example, after installing Node.js inside the Ubuntu container, you now have a container running off an image, but the container is different from the image you used to create it. But you might want to reuse this Node.js container as the basis for new images later.
- Then stop the container.
- After the ubuntu container with the node.js package has been stopped, you can commit the changes to a new Docker image instance using the following command.

```
docker commit -m "added node.js" -a "Vic" e602dd6d84c4 vic/ubuntu_with_node.js
```

# 7. Committing Changes in a Container

## creating a new Docker Image (2/2)

- The `-m` switch is for the commit message that helps you and others know what changes you made, while `-a` is used to specify the author. The `container_id` is `e602dd6d84c4`. Unless you created additional repositories on Docker Hub, the repository is usually your Docker Hub username.
- When you commit an image, the new image is saved **locally** on your computer.
- Listing the Docker images again will show the new image, as well as the old one that it was derived from:

`docker images`

| REPOSITORY              | TAG    | IMAGE ID     | CREATED       | SIZE   |
|-------------------------|--------|--------------|---------------|--------|
| vic/ubuntu_with_node.js | latest | 1f50c25ccd7a | 2 minutes ago | 220MB  |
| ubuntu                  | latest | 94e814e2efa8 | 13 days ago   | 88.9MB |
| hello-world             | latest | fce289e99eb9 | 2 months ago  | 1.84kB |

# 8. Change name of Docker Image

- The `docker tag` command copy a given docker image to a new docker image.
- **`docker tag oldname:oldtag newname:newtag`**
- **To rename a docker image we must create a copy of the original image and then delete the original image**, as in the following example.

- Example:

```
vic@lubuntu:~/REPOSITORY
mymongodb
```

| docker images |              |                |       |  |
|---------------|--------------|----------------|-------|--|
| TAG           | IMAGE ID     | CREATED        | SIZE  |  |
| latest        | ee8f3e38f629 | 17 minutes ago | 984MB |  |

```
vic@lubuntu:~/REPOSITORY
mymongodb
```

```
vic@lubuntu:~/REPOSITORY
mymongodb_1
```

```
vic@lubuntu:~/REPOSITORY
mymongodb
mymongodb_1
```

| docker images |              |                |       |  |
|---------------|--------------|----------------|-------|--|
| TAG           | IMAGE ID     | CREATED        | SIZE  |  |
| latest        | ee8f3e38f629 | 17 minutes ago | 984MB |  |
| latest        | bb9c4a59c723 | 1 minutes ago  | 984MB |  |

```
vic@lubuntu:~/REPOSITORY
mymongodb
```

```
vic@lubuntu:~/REPOSITORY
mymongodb_1
```

| docker images |              |               |       |  |
|---------------|--------------|---------------|-------|--|
| TAG           | IMAGE ID     | CREATED       | SIZE  |  |
| latest        | bb9c4a59c723 | 1 minutes ago | 984MB |  |

# 9. Pushing a Docker image to a Docker repository (1/2)

- The next logical step after creating a new image from an existing image is to share it with a select few of your friends, the whole world on Docker Hub, or other Docker registry that you have access to.
- To push an image to Docker Hub or any other Docker registry, you must have an account there.
- This example shows you how to push a Docker image to Docker Hub.
- Before to do so, you needs to create your own private Docker registry.
  - Create a dockerId at <https://id.docker.com>
- To push your image, first log into Docker Hub.

```
docker login -u docker-registry-username
docker login -u vittorioghini
```
- You'll be prompted to authenticate using your Docker Hub password. If you specified the correct password, authentication should succeed.

# 9. Pushing a Docker image to a Docker repository (2/2)

- Then you may push your own image using:  
`docker push docker-registry-username/docker-image-name`
- As an example, to push the ubuntu-nodejs image to the vittorio.ghini repository, the command would be:

```
docker push vittorioghini/ubuntu_with_node.js
```

- 
- The process may take some time to complete as it uploads the images.

```
docker push vittorioghini/ubuntu_with_node.js
```

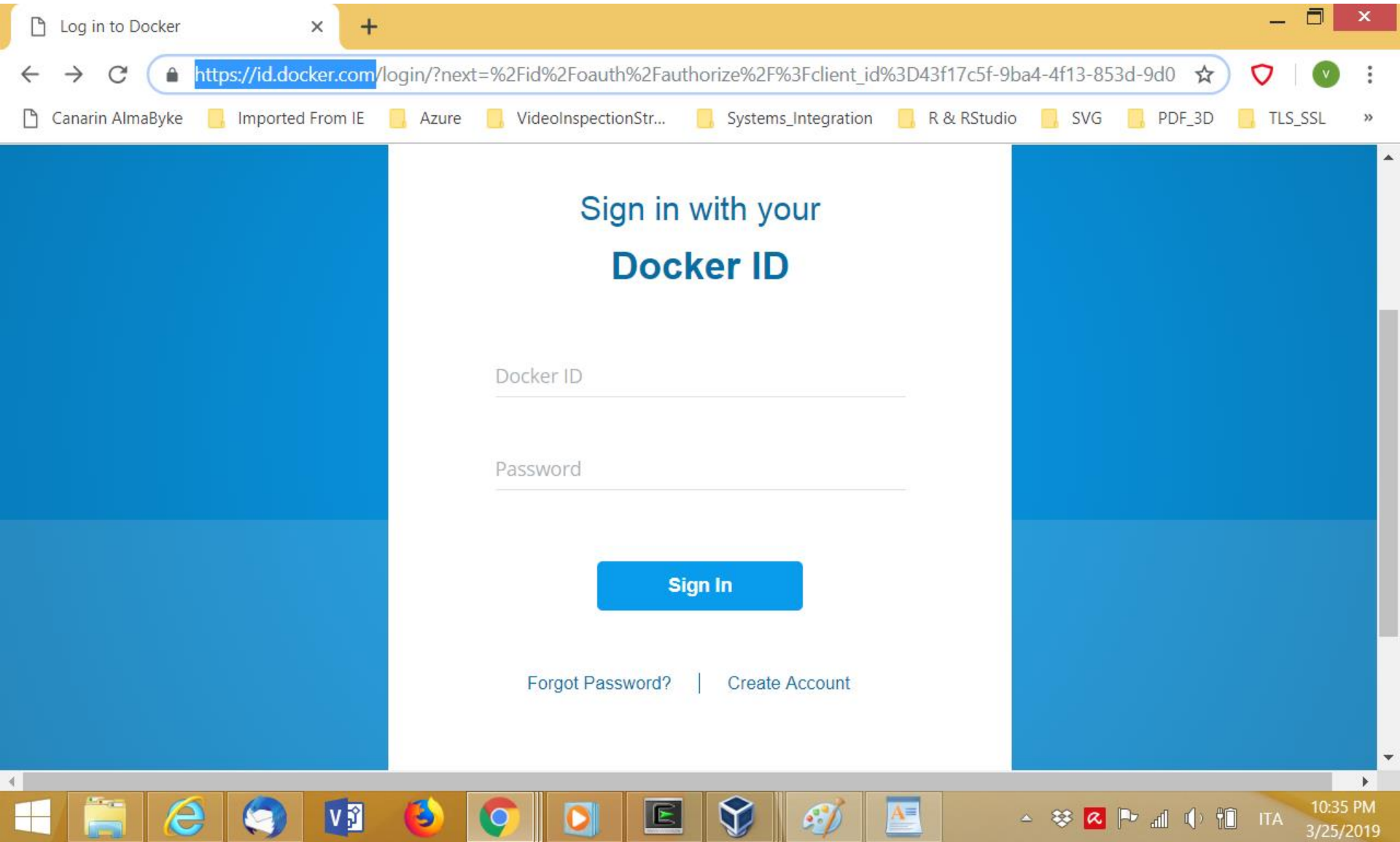
The push refers to repository [docker.io/vittorioghini/ubuntu\_with\_node.js]

e14b35a423b0: Pushing [=====] 79.88MB/131.3MB

b57c79f4a9f3: Pushed

- After pushing an image to a registry, it should be listed on your account's dashboard, like that show in the image below.
- You can now use `docker pull vittorioghini/ubuntu_with_node.js` to pull the image to a new machine and use it to run a new container.

# Docker Hub repository



# Docker Hub repository

Docker Hub

https://hub.docker.com/?ref=login

Canarin AlmaByke Imported From IE Azure VideoInspectionStr... Systems\_Integration R & RStudio SVG PDF\_3D TLS\_SSL

**dockercon19** Join your peers at DockerCon in San Francisco: April 29th – May 2nd. [For a limited time: get 10% off with code: DockerID](#)

dockerhub Search for great content (e.g., mysql) Explore Repositories Organizations Get Help vittorioghini

vittorioghini Search by repository name... **Create Repository +**

vittorioghini / **ubuntu\_with\_node.js** ☆ 0 ↓ 1 PUBLIC  
Updated 11 minutes ago

 Tip: Not finding your repository? Try switching namespace via the top left dropdown.

**Create an Organization**  
Manage Docker Hub repositories with your team

Windows Taskbar: File Explorer, Edge, Teams, VS Code, Firefox, Chrome, VLC, Docker Desktop, Paint, Word, System Tray: Network, Volume, ITA, 10:33 PM 3/25/2019